

Programmazione a oggetti

Classi e Oggetti

Introduzione alla programmazione orientata agli oggetti con classi e oggetti in C++.

Cosa sono le classi e gli oggetti?

Immagina di dover gestire le informazioni di una classe scolastica con 30 studenti. Ogni studente ha un nome, un'età e dei voti.

Senza classi, avresti 30 variabili per i nomi, 30 per le età, 30 per i voti — un caos totale. Con le classi, puoi creare un "modello Studente" e poi creare 30 studenti, ognuno con i suoi dati ben organizzati.

Classe vs Oggetto: lo stampo e il prodotto

Pensa a una **classe** come allo **stampo per fare i biscotti**. Lo stampo definisce la forma — ma non è un biscotto.

Un **oggetto** è il biscotto vero, creato usando quello stampo. Puoi usare lo stesso stampo per fare decine di biscotti, ognuno diverso (con cioccolato, con glassa, ecc.).

- **Classe** = lo stampo (il progetto)
- **Oggetto** = il prodotto concreto (l'istanza)

Come si definisce una classe

```
class Cane {  
public:  
    string nome;    // caratteristiche (dati)  
    string razza;  
    int eta;  
  
    void abbaia() {                                // azioni (funzioni)  
        cout << nome << " dice: Bau!" << endl;  
    }  
  
    void descrivi() {  
        cout << nome << " è un/a " << razza  
            << " di " << eta << " anni." << endl;  
    }  
};
```

La parola **public:** significa che questi dati e queste funzioni sono accessibili dall'esterno della classe.

Come si creano oggetti

Una volta definita la classe, puoi creare quanti oggetti vuoi:

```
Cane c1;                // crea un cane (oggetto vuoto)  
c1.nome = "Fido";        // imposta i dati con il punto .  
c1.razza = "Labrador";  
c1.eta = 3;  
  
Cane c2;  
c2.nome = "Rex";  
c2.razza = "Pastore Tedesco";  
c2.eta = 5;  
  
c1.abbaia();             // Fido dice: Bau!  
c2.descrivi();           // Rex è un/a Pastore Tedesco di 5 anni.
```

Il punto **.** si usa per accedere sia ai dati che alle funzioni di un oggetto.

Un esempio più completo: Rettangolo

```
class Rettangolo {
public:
    double base;
    double altezza;

    // Funzione che calcola l'area
    double area() {
        return base * altezza;
    }

    // Funzione che calcola il perimetro
    double perimetro() {
        return 2 * (base + altezza);
    }
};

int main() {
    Rettangolo r;          // crea un rettangolo
    r.base = 5.0;           // imposta i dati
    r.altezza = 3.0;

    cout << "Area: " << r.area() << endl;          // 15
    cout << "Perimetro: " << r.perimetro() << endl; // 16

    return 0;
}
```

La cosa bella è che `area()` usa automaticamente `base` e `altezza` dell'oggetto su cui viene chiamata. Non devi passarle come parametri.

Differenza tra `class` e `struct`

In C++, `class` e `struct` sono quasi identiche. L'unica differenza:

- In `struct` tutto è **pubblico** per default (accessibile dall'esterno)
- In `class` tutto è **privato** per default (non accessibile dall'esterno)

In pratica, si usa `struct` per semplici raggruppamenti di dati, e `class` per oggetti più complessi con logica interna.

Passare oggetti alle funzioni

Puoi passare un oggetto a una funzione come qualsiasi altra variabile. Per evitare di copiarlo, usa `const &` :

```
// Legge i dati del cane senza modificarli
void stampaInfoCane(const Cane& c) {
    cout << "Nome: " << c.nome << endl;
    cout << "Razza: " << c.razza << endl;
    cout << "Eta: " << c.eta << " anni" << endl;
}

int main() {
    Cane fido;
    fido.nome = "Fido";
    fido.razza = "Labrador";
    fido.eta = 3;

    stampaInfoCane(fido); // passa il cane alla funzione
    return 0;
}
```

Esempio pratico: classe Studente

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

class Studente {
public:
    string nome;
    vector<int> voti;    // lista di voti

    // Aggiunge un voto alla lista
    void aggiungiVoto(int voto) {
        voti.push_back(voto);
    }

    // Calcola la media dei voti
    double calcolaMedia() {
        if (voti.empty()) return 0.0;
        int somma = 0;
        for (int v : voti) {
            somma += v;
        }
        return (double)somma / voti.size();
    }

    // Stampa la scheda completa dello studente
    void stampaScheda() {
        cout << "Studente: " << nome << endl;
        cout << "Voti: ";
        for (int v : voti) {
            cout << v << " ";
        }
        cout << endl;
        cout << "Media: " << calcolaMedia() << endl;
    }
};

int main() {
    Studente alice;
    alice.nome = "Alice";
    alice.aggiungiVoto(8);
    alice.aggiungiVoto(9);
    alice.aggiungiVoto(7);
    alice.aggiungiVoto(10);
}
```

```
    alice.stampaScheda();  
  
    return 0;  
}
```

Output:

```
Studente: Alice  
Voti: 8 9 7 10  
Media: 8.5
```

Costruttori

Come usare i costruttori per inizializzare gli oggetti in C++.

Cos'è un costruttore?

Ogni volta che crei un oggetto, i suoi campi inizialmente contengono valori casuali. Se dimentichi di impostarli prima di usarli, il programma si comporta in modo imprevedibile.

Il **costruttore** risolve questo: è una funzione speciale che viene chiamata **automaticamente** ogni volta che crei un oggetto, e il suo compito è impostare i valori iniziali. Come il modulo di registrazione che compili quando apri un conto in banca.

Come funziona un costruttore

Un costruttore ha tre caratteristiche speciali:

1. Ha lo **stesso nome della classe**
2. **Non ha tipo di ritorno** (nemmeno `void`)
3. Viene chiamato **automaticamente** alla creazione dell'oggetto

```
class Persona {
public:
    string nome;
    int eta;

    // Questo è il costruttore (stesso nome della classe, nessun tipo di
    ritorno)
    Persona() {
        nome = "Sconosciuto"; // imposta valori di default
        eta = 0;
    }
};

int main() {
    Persona p; // il costruttore viene chiamato qui, automaticamente
    cout << p.nome << ", " << p.eta << endl; // Sconosciuto, 0
    return 0;
}
```

Costruttore con parametri

Puoi passare dati al costruttore per inizializzare l'oggetto con valori specifici:

```

class Persona {
public:
    string nome;
    int eta;

    // Costruttore con parametri
    Persona(string n, int e) {
        nome = n;
        eta = e;
    }
};

int main() {
    Persona p1("Alice", 16);    // crea Alice di 16 anni
    Persona p2("Bob", 17);      // crea Bob di 17 anni

    cout << p1.nome << ", " << p1.eta << endl;    // Alice, 16
    cout << p2.nome << ", " << p2.eta << endl;    // Bob, 17
    return 0;
}

```

La lista di inizializzazione

Un modo più efficiente di impostare i campi nel costruttore è la **lista di inizializzazione**, che si scrive dopo i due punti

:

```

class Rettangolo {
private:
    double base;
    double altezza;

public:
    // base(b) significa: inizializza base con il valore di b
    Rettangolo(double b, double a) : base(b), altezza(a) {
        // il corpo può restare vuoto
    }

    double area() const { return base * altezza; }
};

```

È preferita perché inizializza i campi direttamente, senza prima crearli vuoti e poi assegnarli.

Più costruttori per la stessa classe

Puoi avere più costruttori con parametri diversi — il compilatore sceglie quello giusto in base a come crei l'oggetto:

```
class Punto {
public:
    double x;
    double y;

    // Costruttore senza parametri: crea il punto nell'origine
    Punto() : x(0.0), y(0.0) {}

    // Costruttore con un valore: stessa coordinata per x e y
    Punto(double v) : x(v), y(v) {}

    // Costruttore con due valori
    Punto(double x, double y) : x(x), y(y) {}
};

int main() {
    Punto p1;           // (0, 0) – usa il primo costruttore
    Punto p2(5.0);       // (5, 5) – usa il secondo costruttore
    Punto p3(3.0, 4.0);  // (3, 4) – usa il terzo costruttore
    return 0;
}
```

Valori di default nei parametri

Puoi dare valori predefiniti ai parametri del costruttore, così non sei obbligato a fornirli tutti:

```

class Cane {
public:
    string nome;
    string razza;
    int eta;

    // Se non specifichi razza o eta, usano i valori di default
    Cane(string n, string r = "Meticcio", int e = 0)
        : nome(n), razza(r), eta(e) {}

    void descrivi() {
        cout << nome << " (" << razza << ", " << eta << " anni)" << endl;
    }
};

int main() {
    Cane d1("Fido", "Labrador", 3);    // tutti i parametri
    Cane d2("Rex", "Pastore");          // eta usa il default (0)
    Cane d3("Luna");                   // razza e eta usano i default

    d1.descrivi(); // Fido (Labrador, 3 anni)
    d2.descrivi(); // Rex (Pastore, 0 anni)
    d3.descrivi(); // Luna (Meticcio, 0 anni)
    return 0;
}

```

Il distruttore: il contrario del costruttore

Come il costruttore crea e inizializza, il **distruttore** fa la pulizia quando l'oggetto non serve più. Si definisce con il simbolo `~` davanti al nome della classe:

```

class Risorsa {
public:
    Risorsa() {
        cout << "Risorsa creata" << endl;
    }

    ~Risorsa() {
        cout << "Risorsa distrutta" << endl;
    }
};

int main() {
    cout << "Inizio" << endl;
    {
        Risorsa r;                // costruttore chiamato qui
        cout << "In uso" << endl;
    }                             // qui r esce dal blocco → distruttore
    chiamato
    cout << "Fine" << endl;
    return 0;
}

```

Output:

```

Inizio
Risorsa creata
In uso
Risorsa distrutta
Fine

```

Il distruttore è utile soprattutto per liberare risorse (memoria, file aperti, connessioni di rete) quando l'oggetto viene eliminato.

Esempio pratico: conto bancario

```
#include <iostream>
#include <string>
using namespace std;

class ContoBancario {
private:
    string intestatario;
    double saldo;

public:
    // Costruttore: apre il conto
    ContoBancario(string nome, double saldoIniziale = 0.0)
        : intestatario(nome), saldo(saldoIniziale) {
        cout << "Conto aperto per " << intestatario << endl;
    }

    // Distruttore: chiude il conto
    ~ContoBancario() {
        cout << "Conto di " << intestatario << " chiuso." << endl;
    }

    void deposita(double importo) { saldo += importo; }
    double getSaldo() const { return saldo; }
    string getNome() const { return intestatario; }
};

int main() {
    ContoBancario c1("Alice", 1000.0); // saldo iniziale 1000
    ContoBancario c2("Bob");           // saldo iniziale 0 (default)

    c1.deposita(500.0);
    cout << c1.getNome() << ": " << c1.getSaldo() << " euro" << endl;
    cout << c2.getNome() << ": " << c2.getSaldo() << " euro" << endl;

    return 0; // qui i distruttori vengono chiamati automaticamente
}
```

Incapsulamento

Come usare `public` e `private` per proteggere i dati nelle classi C++.

Cos'è l'incapsulamento?

Immagina un distributore automatico. Puoi premere i pulsanti (interfaccia pubblica), ma non puoi mettere le mani nel meccanismo interno (dati privati). Questo è l'**incapsulamento**: nascondere i dettagli interni e offrire solo un accesso controllato dall'esterno.

In pratica: i dati di una classe vengono dichiarati **privati**, così nessuno può modificarli direttamente dall'esterno. Per accedervi, si usano metodi pubblici appositamente creati.

Il problema senza incapsulamento

Quando i dati sono pubblici, chiunque può modificarli — anche con valori assurdi:

```
class Eta {
public:
    int valore;    // pubblico: chiunque può modificarlo
};

int main() {
    Eta e;
    e.valore = -5;    // nessuno impedisce valori assurdi!
    e.valore = 9999;  // nessun controllo!
    return 0;
}
```

La soluzione: dati privati, metodi pubblici

```
class Eta {
private:
    int valore;    // privato: nessuno può accedervi direttamente
                  // dall'esterno

public:
    // Setter: imposta il valore, ma solo se è valido
    void setValore(int v) {
        if (v >= 0 && v <= 120) {
            valore = v;
        } else {
            cout << "Eta non valida!" << endl;
        }
    }

    // Getter: restituisce il valore corrente
    int getValore() const {
        return valore;
    }
};

int main() {
    Eta e;
    e.setValore(16);    // OK
    e.setValore(-5);    // "Eta non valida!" – il controllo blocca il
    valore
    e.setValore(9999);  // "Eta non valida!"

    cout << e.getValore() << endl;    // 16

    // e.valore = 5;    // ERRORE di compilazione: valore è privato!
    return 0;
}
```

I tre livelli di accesso

In C++ ci sono tre parole chiave per controllare chi può accedere ai dati:

Parola chiave	Chi può accedere
<code>public</code>	Tutti — dall'interno della classe e dall'esterno
<code>private</code>	Solo dall'interno della classe
<code>protected</code>	Dalla classe e dalle classi che la estendono (ereditarietà)

Esempio completo: classe Persona

```
#include <iostream>
#include <string>
using namespace std;

class Persona {
private:
    string nome;    // dati privati
    int eta;
    string email;

public:
    // Costruttore: usa i setter per validare i valori fin dall'inizio
    Persona(string n, int e, string em) {
        nome = n;
        setEta(e);    // passa attraverso il setter
        email = em;
    }

    // Getter: leggono i dati privati (sola lettura)
    string getName() const { return nome; }
    int getEta() const { return eta; }
    string getEmail() const { return email; }

    // Setter con validazione
    void setEta(int e) {
        if (e >= 0 && e <= 150) {
            eta = e;
        } else {
            cout << "Eta non valida!" << endl;
            eta = 0;
        }
    }

    void setEmail(string em) {
        if (em.find('@') != string::npos) {    // deve contenere @
            email = em;
        } else {
            cout << "Email non valida!" << endl;
        }
    }

    // Metodo per stampare le informazioni
    void stampa() const {
        cout << "Nome: " << nome << endl;
    }
}
```



```

        cout << "Eta: " << eta << endl;
        cout << "Email: " << email << endl;
    }
};

int main() {
    Persona p("Alice", 16, "alice@esempio.it");
    p.stampa();

    p.setEta(17); // OK
    p.setEmail("non-una-email"); // Email non valida!
    p.setEmail("alice.nuovo@esempio.it"); // OK

    cout << "Nuova eta: " << p.getEta() << endl;
    cout << "Nuova email: " << p.getEmail() << endl;

    return 0;
}

```

Il vantaggio nascosto: libertà di cambiare l'interno

L'incapsulamento ti permette di modificare come funziona internamente una classe **senza dover cambiare il codice che la usa**:

```

// Versione 1: memorizza la temperatura in Celsius
class Termometro {
private:
    double celsius;
public:
    void setTemperatura(double t) { celsius = t; }
    double getTemperatura() const { return celsius; }
};

// Versione 2: internamente usa Kelvin, ma l'interfaccia è identica!
// Chi usa questa classe non sa (e non deve sapere) del cambiamento
// interno.
class Termometro {
private:
    double kelvin; // cambiato internamente
public:
    void setTemperatura(double celsius) { kelvin = celsius + 273.15; }
    double getTemperatura() const { return kelvin - 273.15; }
};

```

Chi scrive `t.setTemperatura(100)` non deve cambiare nulla — l'interfaccia è la stessa.

Ricorda: nelle `class`, tutto è privato per default

```
class Esempio {  
    int x;    // privato per default (nessun public/private specificato)  
  
public:  
    int y;    // pubblico (esplicitamente)  
};
```

È buona abitudine specificare sempre `public:` e `private:` per chiarezza, anche se tecnicamente non sarebbe necessario.

Enumerazioni (enum)

Come usare le enumerazioni in C++ per definire insiemi di costanti con nome.

Cos'è un enum?

Supponi di dover rappresentare i giorni della settimana nel tuo programma. Potresti usare i numeri 0, 1, 2... ma chi si ricorda che 3 è giovedì?

Un'enumerazione (`enum`) ti permette di dare nomi significativi a un insieme fisso di valori. Invece di `3`, scrivi `GIOVEDI`. Il codice diventa leggibile come l'italiano.

Definire un enum

```
enum NomeEnum {  
    VALORE1,  
    VALORE2,  
    VALORE3  
};
```

Esempio concreto:

```
enum GiornoSettimana {  
    LUNEDI,  
    MARTEDI,  
    MERCOLEDI,  
    GIOVEDI,  
    VENERDI,  
    SABATO,  
    DOMENICA  
};
```

Per convenzione, i valori si scrivono tutti in **MAIUSCOLO**.

Usare un enum

```
GiornoSettimana oggi = MERCOLEDI;    // dichiara una variabile di tipo  
GiornoSettimana  
  
if (oggi == SABATO || oggi == DOMENICA) {  
    cout << "Weekend!" << endl;  
} else {  
    cout << "Giorno lavorativo." << endl;  
}
```

I valori numerici nascosti

Internamente, ogni valore dell'enum è un numero. Per default, il primo vale 0, il secondo 1, e così via:

```
enum Stagione {  
    PRIMAVERA, // 0  
    ESTATE,    // 1  
    AUTUNNO,   // 2  
    INVERNO    // 3  
};  
  
Stagione s = ESTATE;  
cout << s << endl; // stampa 1 (il numero interno)
```

Puoi assegnare valori specifici:

```
enum Mese {  
    GENNAIO = 1, // parte da 1 invece di 0  
    FEBBRAIO,    // 2  
    MARZO,       // 3  
    // ...  
    DICEMBRE = 12  
};  
  
Mese m = MARZO;  
cout << m << endl; // 3
```

Abbinare enum e switch

enum e switch si abbinano perfettamente — il codice è chiaro e privo di ambiguità:

```

enum Semaforo {
    ROSSO,
    GIALLO,
    VERDE
};

void istruzione(Semaforo s) {
    switch (s) {
        case ROSSO:
            cout << "STOP!" << endl;
            break;
        case GIALLO:
            cout << "Attenzione, rallentare." << endl;
            break;
        case VERDE:
            cout << "Via libera!" << endl;
            break;
    }
}

int main() {
    istruzione(ROSSO);    // STOP!
    istruzione(VERDE);    // Via libera!
    return 0;
}

```

enum class : la versione moderna e più sicura

Il C++ moderno introduce `enum class`, che evita un problema degli enum classici: i nomi possono andare in conflitto tra enum diversi.

```
// Problema con enum classico:
enum Colore { ROSSO, VERDE, BLU };
enum Frutto { ROSSO, VERDE, GIALLO }; // ERRORE: ROSSO è già definito!

// Soluzione con enum class:
enum class Colore { ROSSO, VERDE, BLU };
enum class Frutto { ROSSO, VERDE, GIALLO }; // nessun conflitto!

int main() {
    Colore c = Colore::ROSSO; // devi specificare il nome dell'enum
    Frutto f = Frutto::ROSSO; // completamente separato da Colore::ROSSO

    if (c == Colore::ROSSO) {
        cout << "Il colore è rosso" << endl;
    }
    return 0;
}
```

Con `enum class` devi sempre scrivere `NomeEnum::VALORE` — è più lungo, ma non ci sono mai ambiguità. È la scelta preferita nel C++ moderno.

Convertire tra `enum` e numeri

```
enum Livello { FACILE, MEDIO, DIFFICILE };

Livello l = MEDIO;
int n = static_cast<int>(l); // da enum a numero → 1
cout << n << endl;

Livello l2 = static_cast(2); // da numero a enum → DIFFICILE
```

Esempio pratico: sistema di voti

```
#include <iostream>
using namespace std;

enum class Giudizio {
    OTTIMO,
    BUONO,
    SUFFICIENTE,
    INSUFFICIENTE
};

// Converte un voto numerico nel giudizio corrispondente
Giudizio classificaVoto(int voto) {
    if (voto >= 9) return Giudizio::OTTIMO;
    if (voto >= 7) return Giudizio::BUONO;
    if (voto >= 6) return Giudizio::SUFFICIENTE;
    return Giudizio::INSUFFICIENTE;
}

// Stampa il giudizio in parole
void stampaGiudizio(Giudizio g) {
    switch (g) {
        case Giudizio::OTTIMO:      cout << "Ottimo!" << endl;
        break;
        case Giudizio::BUONO:       cout << "Buono" << endl;
        break;
        case Giudizio::SUFFICIENTE: cout << "Sufficiente" << endl;
        break;
        case Giudizio::INSUFFICIENTE: cout << "Insufficiente" << endl;
        break;
    }
}

int main() {
    int voti[] = {9, 7, 5, 10, 6};
    for (int voto : voti) {
        cout << "Voto " << voto << ": ";
        stampaGiudizio(classificaVoto(voto));
    }
    return 0;
}
```

Output:

Voto 9: Ottimo!
Voto 7: Buono
Voto 5: Insufficiente
Voto 10: Ottimo!
Voto 6: Sufficiente

Ereditarietà in C++

Introduzione all'ereditarietà in C++, come creare classi derivate da classi esistenti.

Cos'è l'ereditarietà?

Immagina di dover creare le classi `Cane`, `Gatto` e `Coniglio`. Tutte e tre hanno un nome, un'età, mangiano e dormono. Riscrivere le stesse cose tre volte è una perdita di tempo.

L'**ereditarietà** risolve questo: scrivi una classe `Animale` con le caratteristiche comuni, e poi `Cane`, `Gatto` e `Coniglio` **ereditano** tutto da essa, aggiungendo solo quello che hanno in più.

Come in natura: un cane è un animale. Ha tutto quello che ha un animale, più le caratteristiche proprie del cane.

Come funziona l'ereditarietà

La classe "figlia" (derivata) eredita tutto dalla classe "madre" (base):


```

// Classe base (la "madre")
class Animale {
public:
    string nome;
    int eta;

    void mangia() {
        cout << nome << " sta mangiando." << endl;
    }

    void dormi() {
        cout << nome << " sta dormendo." << endl;
    }
};

// Classe derivata (la "figlia") – eredita da Animale
class Cane : public Animale {
public:
    string razza;    // campo aggiuntivo specifico del Cane

    void abbaia() {
        cout << nome << " dice: Bau!" << endl;    // usa il campo di
Animale!
    }
};

// Un'altra classe derivata
class Gatto : public Animale {
public:
    bool viveInCasa;

    void faSeLeFusa() {
        cout << nome << " fa le fusa." << endl;
    }
};

```

Usare le classi derivate

```
int main() {  
    Cane fido;  
    fido.nome = "Fido";      // ereditato da Animale  
    fido.eta = 3;  
    fido.razza = "Labrador"; // specifico di Cane  
  
    fido.mangia();           // metodo ereditato da Animale  
    fido.abbaia();           // metodo proprio di Cane  
  
    Gatto luna;  
    luna.nome = "Luna";  
    luna.dormi();            // metodo ereditato  
    luna.faSeLeFusa();       // metodo proprio di Gatto  
  
    return 0;  
}
```

Il costruttore nelle classi derivate

Quando crei un oggetto `Cane`, il C++ chiama prima il costruttore di `Animale` e poi quello di `Cane`. Per passare dati al costruttore della classe madre, usa `:` nella lista di inizializzazione:

```

class Animale {
public:
    string nome;
    int eta;

    Animale(string n, int e) : nome(n), eta(e) {
        cout << "Animale creato: " << nome << endl;
    }
};

class Cane : public Animale {
public:
    string razza;

    // Chiama il costruttore di Animale passandogli n e e
    Cane(string n, int e, string r) : Animale(n, e), razza(r) {
        cout << "Cane creato: " << nome << " (" << razza << ")" << endl;
    }
};

int main() {
    Cane fido("Fido", 3, "Labrador");
    // Stampa:
    // Animale creato: Fido
    // Cane creato: Fido (Labrador)
    return 0;
}

```

Sovrascrivere un metodo della classe madre

Una classe figlia può **ridefinire** un metodo della madre per dargli un comportamento diverso:

```

class Animale {
public:
    string nome;

    void emettiSuono() {
        cout << nome << " emette un suono generico." << endl;
    }
};

class Cane : public Animale {
public:
    // Sovrascrive il metodo della classe madre
    void emettiSuono() {
        cout << nome << " dice: Bau!" << endl;
    }
};

class Gatto : public Animale {
public:
    void emettiSuono() {
        cout << nome << " dice: Miao!" << endl;
    }
};

int main() {
    Animale a; a.nome = "Generico";
    Cane c;    c.nome = "Fido";
    Gatto g;   g.nome = "Luna";

    a.emettiSuono(); // emette un suono generico
    c.emettiSuono(); // Bau!
    g.emettiSuono(); // Miao!
    return 0;
}

```

Chiamare il metodo della classe madre

Se hai bisogno di usare anche il metodo originale della madre dentro quello della figlia:

```

class Animale {
public:
    virtual void descrivi() {
        cout << "Sono un animale." << endl;
    }
};

class Cane : public Animale {
public:
    void descrivi() {
        Animale::descrivi();    // chiama prima il metodo della madre
        cout << "Sono un cane." << endl;    // poi aggiunge qualcosa
    }
};

```

protected : il livello intermedio

private significa che nemmeno le classi figlie possono accedere al dato. A volte vuoi qualcosa di più accessibile, ma non completamente pubblico: usa **protected** :

```

class Animale {
protected:
    string nome;    // le classi figlie possono usarlo
private:
    int codiceInterno;    // le classi figlie NON possono vederlo
public:
    Animale(string n) : nome(n) {}
};

class Cane : public Animale {
public:
    Cane(string n) : Animale(n) {}

    void abbaia() {
        cout << nome << " dice: Bau!";    // OK: nome è protected
        cout << codiceInterno;    // ERRORE: è private
    }
};

```

Catene di ereditarietà

Puoi creare gerarchie con più livelli:

```
class Veicolo {
public:
    int velocitaMassima;
    void muoviti() { cout << "Mi muovo." << endl; }
};

class Auto : public Veicolo {
public:
    int numeroPosti;
};

class SportCar : public Auto {
public:
    bool haIlTurbo;
    void sgomma() { cout << "Sgommata!" << endl; }
};
```

`SportCar` eredita da `Auto`, che eredita da `Veicolo`. Quindi `SportCar` ha `velocitaMassima`, `numeroPosti`, `haIlTurbo`, più i metodi `muoviti()` e `sgomma()`.

Metodi

Come definire e usare i metodi nelle classi C++.

Cosa sono i metodi?

Un oggetto non è solo un contenitore di dati — può anche fare cose. I **metodi** sono le azioni che un oggetto sa compiere.

Un oggetto `Cane` ha dati (nome, razza, età) ma può anche abbaiare, mangiare, correre. Un oggetto `ContoBancario` ha un saldo, ma può anche ricevere depositi e prelievi.

I metodi sono semplicemente **funzioni scritte dentro una classe**.

Definire metodi dentro la classe

```
class Cerchio {
public:
    double raggio;

    // Metodo che calcola l'area
    double area() {
        return 3.14159 * raggio * raggio;
    }

    // Metodo che calcola il perimetro
    double perimetro() {
        return 2 * 3.14159 * raggio;
    }
};

int main() {
    Cerchio c;
    c.raggio = 5.0;

    cout << "Area: " << c.area() << endl;          // usa automaticamente
c.raggio
    cout << "Perimetro: " << c.perimetro() << endl;
}
```

Il metodo ha accesso diretto ai dati dell'oggetto su cui viene chiamato.

Separare dichiarazione e definizione

Per classi grandi, puoi dichiarare il metodo dentro la classe e scriverne il corpo fuori, usando `NomeClasse::` :

```

class Cerchio {
public:
    double raggio;

    double area();          // solo la firma (dichiarazione)
    double perimetro();     // solo la firma
};

// Il corpo viene scritto fuori dalla classe
double Cerchio::area() {
    return 3.14159 * raggio * raggio;
}

double Cerchio::perimetro() {
    return 2 * 3.14159 * raggio;
}

```

L'operatore `::` dice "questo metodo appartiene alla classe Cerchio".

Metodi che non modificano i dati: **const**

Se un metodo legge solo i dati senza cambiarli, aggiungere `const` alla fine è una buona abitudine. Serve come promessa al compilatore (e al lettore del codice):

```

class Rettangolo {
public:
    double base;
    double altezza;

    double area() const {          // const = non modifica i dati
        return base * altezza;
    }

    void ridimensiona(double b, double a) { // nessun const = può
        modificare
        base = b;
        altezza = a;
    }
};

```


Se provi a modificare un campo dentro un metodo `const`, il compilatore ti segnala l'errore.

Risolvere conflitti di nome con `this`

Dentro un metodo, `this` è un puntatore speciale che indica "l'oggetto corrente". Si usa raramente, ma è utile quando il nome di un parametro coincide con il nome di un campo:

```
class Persona {  
public:  
    string nome;  
  
    void setName(string nome) {  
        // "nome" da solo si riferisce al parametro  
        // "this->nome" si riferisce al campo della classe  
        this->nome = nome;  
    }  
};
```

Getter e Setter: porte controllate per i dati privati

Quando i dati di una classe sono privati (non accessibili dall'esterno), si usano metodi appositi per leggerli e modificarli:

- **Getter:** legge un dato privato
- **Setter:** modifica un dato privato (può anche validare)

```

class Temperatura {
private:
    double gradi;    // non accessibile dall'esterno

public:
    // Setter: imposta la temperatura, ma solo se valida
    void setGradi(double t) {
        if (t >= -273.15) {    // non può scendere sotto lo zero assoluto
            gradi = t;
        } else {
            cout << "Temperatura non valida!" << endl;
        }
    }

    // Getter: legge la temperatura
    double getGradi() const {
        return gradi;
    }

    // Metodo utile: converte in Fahrenheit
    double inFahrenheit() const {
        return gradi * 9.0/5.0 + 32;
    }
};

int main() {
    Temperatura t;
    t.setGradi(100.0);
    cout << t.getGradi() << "°C = " << t.inFahrenheit() << "°F" << endl;
    // 100°C = 212°F

    t.setGradi(-300.0);    // Temperatura non valida!
    return 0;
}

```

Esempio pratico: conto bancario

```
#include <iostream>
#include <string>
using namespace std;

class ContoBancario {
private:
    string intestatario;
    double saldo;

public:
    // Inizializza il conto
    void apri(string nome, double saldoIniziale) {
        intestatario = nome;
        saldo = saldoIniziale;
    }

    // Aggiunge denaro al saldo
    void deposita(double importo) {
        if (importo > 0) {
            saldo += importo;
            cout << "Depositati " << importo
                 << " euro. Saldo: " << saldo << " euro." << endl;
        }
    }

    // Toglie denaro dal saldo (solo se ci sono fondi)
    bool preleva(double importo) {
        if (importo > 0 && importo <= saldo) {
            saldo -= importo;
            cout << "Prelevati " << importo
                 << " euro. Saldo: " << saldo << " euro." << endl;
            return true;
        }
        cout << "Fondi insufficienti!" << endl;
        return false;
    }

    // Getter per leggere i dati privati
    double getSaldo() const { return saldo; }
    string getIntestatario() const { return intestatario; }
};

int main() {
    ContoBancario conto;
```

```
conto.apri("Alice", 1000.0);

conto.deposita(500.0);    // Depositati 500 euro. Saldo: 1500.
conto.preleva(200.0);    // Prelevati 200 euro. Saldo: 1300.
conto.preleva(2000.0);   // Fondi insufficienti!

cout << "Saldo finale: " << conto.getSaldo() << " euro" << endl;

return 0;
}
```

Polimorfismo

Il concetto di polimorfismo in C++, con funzioni virtuali e override.

Cos'è il polimorfismo?

Immagina di avere una fattoria con cani, gatti e mucche. Vuoi far "parlare" tutti gli animali, uno per uno. Senza il polimorfismo, dovresti scrivere un ciclo separato per ogni tipo di animale.

Con il **polimorfismo**, puoi mettere tutti gli animali in una stessa lista e far sì che ognuno risponda al comando "emetti suono" a modo suo — automaticamente, senza controllare di che tipo è.

Polimorfismo significa "molte forme": lo stesso comando, comportamenti diversi a seconda dell'oggetto.

Il problema senza polimorfismo

```
class Cane {
public:
    void suona() { cout << "Bau!" << endl; }
};

class Gatto {
public:
    void suona() { cout << "Miao!" << endl; }
};

// Senza polimorfismo, serve una funzione per ogni tipo
void faiSuonare(Cane& c) { c.suona(); }
void faiSuonare(Gatto& g) { g.suona(); }
// E se aggiungi Mucca? Devi aggiungere un'altra funzione...
```

La parola chiave **virtual**

Aggiungendo **virtual** davanti a un metodo nella classe madre, dici al C++ di usare il comportamento della classe figlia, non della madre:

```

class Animale {
public:
    string nome;

    virtual void emettiSuono() { // virtual = usa il metodo della figlia
        cout << nome << " emette un suono." << endl;
    }
};

class Cane : public Animale {
public:
    void emettiSuono() override { // override = sovrascrive il metodo
base
        cout << nome << " dice: Bau!" << endl;
    }
};

class Gatto : public Animale {
public:
    void emettiSuono() override {
        cout << nome << " dice: Miao!" << endl;
    }
};

```

La parola **override** non è obbligatoria, ma è una buona abitudine: il compilatore ti avvisa se stai cercando di sovrascrivere un metodo che non esiste nella classe madre.

Il polimorfismo in azione

La "magia" si vede quando usi un **puntatore alla classe madre** per gestire oggetti di classi figlie diverse:

```
Animale* a1 = new Cane();    // puntatore ad Animale, ma è un Cane
Animale* a2 = new Gatto();   // puntatore ad Animale, ma è un Gatto

a1->nome = "Fido";
a2->nome = "Luna";

a1->emettiSuono();    // Fido dice: Bau!    (usa il metodo di Cane!)
a2->emettiSuono();    // Luna dice: Miao!    (usa il metodo di Gatto!)

delete a1;
delete a2;
```

Senza `virtual`, entrambi avrebbero chiamato il metodo di `Animale` (suono generico). Con `virtual`, il C++ sa di dover usare il metodo del tipo reale dell'oggetto.

Gestire una lista mista di oggetti

Questo è il punto di forza del polimorfismo: puoi mettere tipi diversi in una stessa lista e trattarli tutti allo stesso modo:

```

#include <iostream>
#include <vector>
using namespace std;

class Animale {
public:
    string nome;
    Animale(string n) : nome(n) {}
    virtual void emettiSuono() = 0;    // obbliga le classi figlie a
    implementarlo
    virtual ~Animale() {}             // distruttore virtuale (importante!)
};

class Cane : public Animale {
public:
    Cane(string n) : Animale(n) {}
    void emettiSuono() override { cout << nome << ": Bau!" << endl; }
};

class Gatto : public Animale {
public:
    Gatto(string n) : Animale(n) {}
    void emettiSuono() override { cout << nome << ": Miao!" << endl; }
};

class Mucca : public Animale {
public:
    Mucca(string n) : Animale(n) {}
    void emettiSuono() override { cout << nome << ": Mu!" << endl; }
};

int main() {
    vector fattoria;
    fattoria.push_back(new Cane("Fido"));
    fattoria.push_back(new Gatto("Luna"));
    fattoria.push_back(new Mucca("Bessie"));
    fattoria.push_back(new Cane("Rex"));

    // Un solo ciclo gestisce tutti i tipi di animali!
    for (Animale* a : fattoria) {
        a->emettiSuono();
    }

    // Libera la memoria
    for (Animale* a : fattoria) {
        delete a;
    }
}

```



```
    return 0;  
}
```

Output:

```
Fido: Bau!  
Luna: Miao!  
Bessie: Mu!  
Rex: Bau!
```

Classi astratte: il modello obbligatorio

Quando scrivi `= 0` dopo un metodo virtuale, quel metodo diventa **puro virtuale**: non ha corpo nella classe madre e ogni classe figlia è obbligata a implementarlo.

Una classe con almeno un metodo puro virtuale è **astratta**: non puoi creare oggetti direttamente da essa.

```

class Forma {
public:
    // Ogni forma DEVE implementare questi due metodi
    virtual double area() const = 0;
    virtual double perimetro() const = 0;

    // Questo metodo usa i precedenti (funziona grazie al polimorfismo)
    void stampa() const {
        cout << "Area: " << area() << ", Perimetro: " << perimetro() <<
endl;
    }
};

class Cerchio : public Forma {
private:
    double raggio;
public:
    Cerchio(double r) : raggio(r) {}
    double area() const override { return 3.14159 * raggio * raggio; }
    double perimetro() const override { return 2 * 3.14159 * raggio; }
};

class Rettangolo : public Forma {
private:
    double base, altezza;
public:
    Rettangolo(double b, double a) : base(b), altezza(a) {}
    double area() const override { return base * altezza; }
    double perimetro() const override { return 2 * (base + altezza); }
};

int main() {
    // Forma f; // ERRORE: Forma è astratta, non puoi creare oggetti diretti

    Cerchio c(5.0);
    Rettangolo r(4.0, 6.0);

    c.stampa(); // Area: 78.5397, Perimetro: 31.4159
    r.stampa(); // Area: 24, Perimetro: 20

    return 0;
}

```

Il distruttore virtuale

Quando usi il polimorfismo con puntatori alla classe madre, dichiara sempre il distruttore della classe madre come `virtual`. Altrimenti, quando elimini un oggetto con `delete`, viene chiamato solo il distruttore della madre — non quello della figlia:

```
class Base {
public:
    virtual ~Base() {           // distruttore virtuale
        cout << "Base distrutta" << endl;
    }
};

class Derivata : public Base {
public:
    ~Derivata() override {
        cout << "Derivata distrutta" << endl;
    }
};

Base* ptr = new Derivata();
delete ptr;    // con virtual: chiama Derivata prima, poi Base
               // senza virtual: chiama solo Base (bug!)
```

Strutture (struct)

Come raggruppare dati correlati in strutture in C++.

Cos'è una struct?

Immagina di dover tenere traccia di 30 studenti. Ogni studente ha un nome, un'età e un voto. Potresti usare tre array separati — ma è un disastro da gestire.

Una **struttura** (`struct`) ti permette di raggruppare tutte le informazioni di uno studente in un unico "pacchetto". Così hai un array di studenti, non tre array separati.

Definire una struttura

```
struct NomeStruttura {  
    tipo1 campo1;  
    tipo2 campo2;  
    // ...  
}; // nota il punto e virgola finale!
```

Esempio concreto:

```
struct Studente {  
    string nome;  
    int eta;  
    double media;  
};
```

Creare e usare una struttura

```
Studente s1;           // crea una variabile di tipo Studente  
  
// Accedi ai campi con il punto .  
s1.nome = "Alice";  
s1.eta = 16;  
s1.media = 8.5;  
  
cout << s1.nome << endl; // Alice  
cout << s1.media << endl; // 8.5
```

Inizializzazione rapida

Puoi impostare tutti i valori in una sola riga:

```
Studente s1 = {"Alice", 16, 8.5}; // ordine: nome, eta, media

// C++11: puoi anche specificare il nome dei campi
Studente s2 = {.nome = "Bob", .eta = 17, .media = 7.2};
```

Strutture come parametri di funzioni

Le strutture si passano alle funzioni esattamente come le variabili normali:

```
struct Punto {
    double x;
    double y;
};

// Calcola la distanza tra due punti
double distanza(Punto p1, Punto p2) {
    double dx = p1.x - p2.x;
    double dy = p1.y - p2.y;
    return sqrt(dx*dx + dy*dy);
}

int main() {
    Punto a = {0.0, 0.0};
    Punto b = {3.0, 4.0};

    cout << "Distanza: " << distanza(a, b) << endl; // 5
    return 0;
}
```

Per modificare una struttura dentro una funzione, passa per riferimento (&):

```

struct Contatore {
    int valore;
    int passi;
};

void incrementa(Contatore& c, int n) {
    c.valore += n;    // modifica l'originale, non una copia
    c.passi++;
}

int main() {
    Contatore c = {0, 0};
    incrementa(c, 10);
    incrementa(c, 5);
    cout << "Valore: " << c.valore << ", Passi: " << c.passi << endl;
    // Valore: 15, Passi: 2
    return 0;
}

```

Array di strutture

Puoi creare array di strutture per gestire più elementi dello stesso tipo:

```

struct Prodotto {
    string nome;
    double prezzo;
    int quantita;
};

int main() {
    Prodotto magazzino[3] = {
        {"Mela", 0.50, 100},
        {"Banana", 0.30, 150},
        {"Arancia", 0.80, 80}
    };

    cout << "Prodotti in magazzino:" << endl;
    for (int i = 0; i < 3; i++) {
        cout << magazzino[i].nome << " - "
            << magazzino[i].prezzo << " euro"
            << " (quantita: " << magazzino[i].quantita << ")" << endl;
    }

    // Calcola il valore totale
    double totale = 0;
    for (int i = 0; i < 3; i++) {
        totale += magazzino[i].prezzo * magazzino[i].quantita;
    }
    cout << "Valore totale: " << totale << " euro" << endl;

    return 0;
}

```

Strutture annidate

Una struttura può contenere un'altra struttura come campo:

```

struct Indirizzo {
    string via;
    string citta;
    string cap;
};

struct Persona {
    string nome;
    int eta;
    Indirizzo indirizzo;    // struttura dentro struttura
};

int main() {
    Persona p;
    p.nome = "Alice";
    p.eta = 16;
    p.indirizzo.via = "Via Roma 10";
    p.indirizzo.citta = "Milano";
    p.indirizzo.cap = "20100";

    cout << p.nome << " vive a " << p.indirizzo.citta << endl;
    return 0;
}

```

Differenza tra struct e class

In C++, `struct` e `class` sono quasi identiche. L'unica differenza:

- In `struct` tutto è **pubblico** per default
- In `class` tutto è **privato** per default

Nella pratica, si usa `struct` per raggruppare dati semplici senza logica, e `class` per oggetti più complessi con comportamenti e dati protetti.

Esempio pratico: registro studenti

```
#include <iostream>
#include <string>
using namespace std;

struct Studente {
    string nome;
    int eta;
    double voto;
};

// Passa per const & per leggere senza copiare
void stampa(const Studente& s) {
    cout << s.nome << " (" << s.eta << " anni) - Voto: " << s.voto << endl;
}

int main() {
    const int N = 3;
    Studente classe[N] = {
        {"Alice", 16, 8.5},
        {"Bob", 17, 7.0},
        {"Carlo", 16, 9.2}
    };

    cout << "=== Registro ===" << endl;
    double somma = 0;
    for (int i = 0; i < N; i++) {
        stampa(classe[i]);
        somma += classe[i].voto;
    }

    cout << "Media della classe: " << somma / N << endl;

    return 0;
}
```

Output:

```
=== Registro ===
Alice (16 anni) - Voto: 8.5
Bob (17 anni) - Voto: 7
Carlo (16 anni) - Voto: 9.2
Media della classe: 8.23333
```